Object Oriented Programming (CSEG1044)
Unit 04 – Lecture 04 Notes
Swing Basics (Components, Containers, Layouts)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Swing overview: `JFrame`, `JPanel`, components vs containers 3

7.2 Layout managers (Flow/Border/Grid) 4

7.3 UI composition: multi-panel skeleton 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: a multi-panel form skeleton **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Main new ideas:** Swing overview: `JFrame`, `JPanel`, components vs containers, Layout managers (Flow/Border/Grid), UI composition: multi-panel skeleton
- **Course thread:** Use Swing Basics (Components, Containers, Layouts) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

2 Lecture Overview

Swing is Java's GUI toolkit for building desktop applications. In this lecture we learn the minimum you need to build a clean UI skeleton:

- top-level window (`JFrame`),
- containers (`JPanel`) and components (`JLabel`, `JButton`, `JTextField`),
- layout managers (Flow/Border/Grid),
- composing multiple panels to create a complete screen.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Classes and packages**: Swing applications still depend on clear classes and package boundaries. Main reference: `Unit 02 / Lecture 03 slides/notes`.
- **Nested helpers**: GUI code often uses callback and helper patterns introduced earlier. Main reference: `Unit 03 / Lecture 01 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Explain Swing component model: `JFrame`, `JPanel`, components vs containers.
- Use layout managers (Flow/Border/Grid) to structure a UI.
- Build and run a small multi-panel UI skeleton.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Swing overview: `JFrame`, `JPanel`, components vs containers → Layout managers (Flow/Border/Grid) → UI composition: multi-panel skeleton.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Component**: UI element like button, label, text field.
- **Container**: holds components (e.g., `JPanel`, `JFrame` content pane).
- **Layout manager**: decides how components are arranged (sizes/positions).
- **Event Dispatch Thread (EDT)**: Swing UI thread (advanced but important for correctness).
- **MVC idea**: separate UI from business logic (we will use this later with JDBC).

7 Detailed Notes

7.1 Swing overview: `JFrame`, `JPanel`, components vs containers

Swing applications usually have:

- a top-level window: `JFrame`,
- inside it, one or more `JPanel` containers,
- inside panels, UI components: `JLabel`, `JTextField`, `JButton`, etc.

Best practice:

- Build UI first (layout + widgets), then add event handling (next lecture).
- Keep UI code separate from core logic (helps testing and maintenance).
- Call `setVisible(true)` at the end, after adding components.

7.2 Layout managers (Flow/Border/Grid)

Swing layouts prevent hard-coded pixel positions and make UI more robust.

FlowLayout:

- Places components left-to-right, wrapping to next line.
- Good for small button rows.

BorderLayout:

- 5 regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, `CENTER`.
- Great for "header + main content + footer" screens.

GridLayout:

- Uniform rows/columns.
- Good for forms (label + field pairs) and simple dashboards.

7.3 UI composition: multi-panel skeleton

A very common pattern for beginners:

- **Root panel**: `BorderLayout`
- **Top panel (NORTH)**: title label
- **Center panel**: form or table
- **Bottom panel (SOUTH)**: action buttons (save, clear, exit)

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

8 Running Project Connection

Use Swing Basics (Components, Containers, Layouts) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.

- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Putting every control directly on one JFrame without structure.
- Hardcoding coordinates instead of using layout managers intentionally.
- Mixing UI code with business logic too early.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

10 Worked Example: a multi-panel form skeleton

```java
import javax.swing.*;
import java.awt.*;

public class SimpleFormUI {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame f = new JFrame("Campus Resource Manager (Skeleton)");
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            JPanel root = new JPanel(new BorderLayout(10, 10));
            root.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

            JLabel title = new JLabel("Add Resource", SwingConstants.CENTER);
            title.setFont(title.getFont().deriveFont(Font.BOLD, 18f));
            root.add(title, BorderLayout.NORTH);

            JPanel form = new JPanel(new GridLayout(3, 2, 8, 8));
            JTextField nameField = new JTextField();
            JTextField typeField = new JTextField();
            JTextField capacityField = new JTextField();
            form.add(new JLabel("Name:")); form.add(nameField);
            form.add(new JLabel("Type:")); form.add(typeField);
            form.add(new JLabel("Capacity:")); form.add(capacityField);
            root.add(form, BorderLayout.CENTER);

            JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
            JButton saveBtn = new JButton("Save");
            JButton clearBtn = new JButton("Clear");
            JButton exitBtn = new JButton("Exit");
            actions.add(saveBtn);
```

```
            actions.add(clearBtn);
            actions.add(exitBtn);
            root.add(actions, BorderLayout.SOUTH);

            f.setContentPane(root);
            f.pack();
            f.setLocationRelativeTo(null); // center on screen
            f.setVisible(true);
        });
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Swing overview: `JFrame`, `JPanel`, components vs containers* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What is the difference between a component and a container?

   Answer: a component is a UI element; a container holds components.

2. Which layout is good for header + main content + footer screens?

   Answer: `BorderLayout`.

3. Why do we use layout managers instead of fixed pixel positions?

   Answer: layouts adapt better to different screen sizes/fonts and reduce UI bugs.

13 Self-Study Practice (no solutions)

1. Add one more field to the form: `Location`.
2. Replace `GridLayout` with `GridBagLayout` (optional advanced) and observe flexibility.
3. Create a second screen panel (e.g., `List Resources`) and switch panels (hint: `CardLayout`).

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why is `pack()` often preferred over `setSize()`?

Sample answer: `pack()` sizes the window based on preferred sizes of components and layout, producing a more consistent UI across systems.

Q2. What is the role of `JPanel`?

Sample answer: `JPanel` is a container used to group and layout components. It helps structure the UI into manageable parts.

Q3. Why do layout managers matter more than pixel coordinates in most Swing code?

Sample answer: Layouts adapt better to resizing, platform differences, and maintenance.